QCert: Verifying and Extending Quantum Circuit Completeness in Agda and Lean

ANONYMOUS AUTHOR(S)

A set of circuit equations is complete if every equation between circuits denoting the same linear map follows from it, and minimal if no equation can be derived from the others. Establishing these properties takes long case analyses spanning hundreds of pages, making the details hard to check by hand. To address this, we extend an existing formalization library to QCert in Agda and Lean. In QCert, humans set up a unified framework of circuits, matrix semantics, and group presentations, and state each paper's main theorem in it. LLM coding agents then write the proofs following the papers' arguments, and the proof assistant checks them. A human therefore need only review each paper's relations, gate semantics, and theorem statement. Using QCert, we first check sixteen published results, fifteen on completeness and one on minimality. They cover discrete gate sets such as Clifford, Clifford+T, and Toffoli–Hadamard, on qubits or qudits of every odd prime dimension. We verify fourteen of them directly and correct an error in each of the other two. We then prove a new result, generalizing the completeness of Toffoli–Hadamard circuits from three to any number of qubits. We expect QCert to help with other open completeness problems, such as those for quantum circuits in higher dimensions.

1 Introduction

Rewriting of quantum circuits is a ubiquitous tool in circuit optimization [36, 44, 45, 55], equivalence checking [56], hardware mapping [53], and reasoning about quantum programs [26]. This relies on an *equational theory* for a family of quantum circuits, a set of rewrite rules that transform a circuit into an equivalent one. Such a theory should be *sound* so that every rewrite preserves the underlying linear map. It should also be *complete* so that any equivalence between circuits can be derived. Additionally, we want it to be *minimal*, with no redundant rules.

Completeness and minimality proofs typically require long case distinctions over gates, relations, and normal forms, whose supporting derivations sometimes span hundreds of pages [20, 26, 38]. In particular, derivations of relations from the rules require intricate circuit manipulations and repeated rewrites, making them prone to human mistakes. For example, circular reasoning can lead us to remove a rule whose derivation still relies on that rule, hidden in a chain of other derivations. Errors can then propagate through dependent derivations, invalidating the entire argument and making the error source hard to trace by hand. A proof assistant would catch such errors.

Our contributions. We present QCert, a framework in Agda [15, 47] and Lean [24] for representing and verifying quantum circuit completeness and minimality results. Figure 1 summarises its workflow: humans check that the formal definitions, relations, and theorem statement faithfully express the source paper; LLM coding agents formalize the proof, and Lean or Agda checks it.

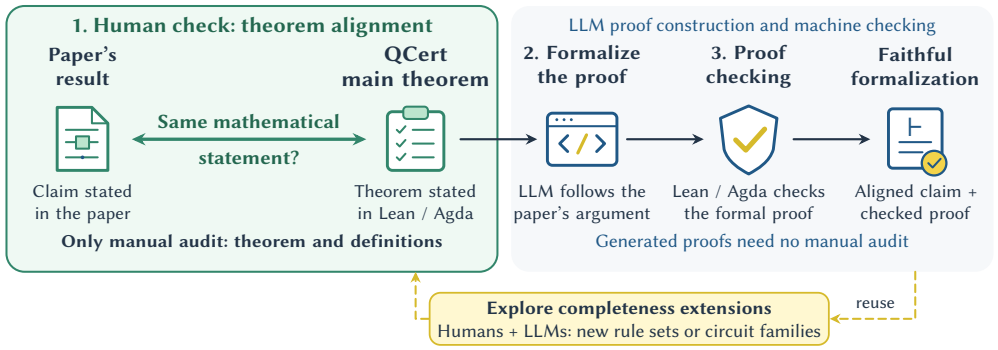


Fig. 1. Using QCert's shared, verified libraries, humans need only check that its main theorem faithfully expresses the paper's result, including the relations, target group, gate matrices, and interpretation. LLM coding agents formalize the source papers' arguments and supply missing proofs or details where needed. Lean or Agda checks the generated proof, so the proof itself needs no human review. The dashed branch shows how humans and LLMs can join forces to explore new completeness results, building on QCert's verified libraries. Each newly obtained result undergoes the same human review and machine checking.

Formalizing proofs of this length by hand would be costly, and LLM coding agents now make it practical. Machine checking alone, however, does not ensure that a formal statement matches its intended claim. For example, Bastounis et al. [6] identify mistranslations in a claimed Navier–Stokes formalization that weakened the statement being verified. QCert therefore leaves the definitions and theorem statements to human review [35].

We use QCert to check sixteen published results, verifying fourteen directly and correcting an error in each of the other two (Section 6). We formalize all sixteen in both Agda and Lean. The Lean version is comparatively easy to obtain from the Agda one, and its point is that it states each theorem over Mathlib's standard definitions, which makes the statements easier to check against the papers (Section 3.3). QCert grew out of our formalization of the qudit Clifford completeness result of Bian, Li et al. [8] (Section 4). That formalization and the library of circuits and presentations it builds on [7] were written largely by hand.

Most of the sixteen results concern restricted Clifford+T circuits or their matrix groups, and QCert reuses the mathematical structure they share. Amy et al. [2] characterise several restricted gate sets by unitary matrices over subrings of $\mathbb{Z}[1/\sqrt{2}, i]$, with ancillas where needed. Figure 2 displays these gate sets, their matrix groups, and multi-level generators. These connections allow QCert to share exact arithmetic, matrix semantics, group constructions, and completeness arguments across formalizations.

QCert also provides a foundation for LLM-assisted exploration of new completeness results. Its shared definitions and verified libraries allow coding agents to build on existing mathematics and automate derivations that would otherwise require substantial manual effort. As a case study, we use this workflow, the dashed branch of Figure 1, to extend Toffoli–Hadamard completeness from three to any number of qubits. Alongside the proofs checked in QCert, we are developing a clear mathematical exposition of the underlying arguments. This case study demonstrates QCert's potential as a research tool: it makes automated proof development practical while grounding confidence in machine checking and human review of the definitions and theorem statements.

Related work. Complete equational theories are known for reversible [27, 37], Clifford [8, 38, 43, 52], and CNOT-dihedral circuits [1], as well as several restricted Clifford+T families [3, 10, 11, 20], and universal quantum circuits [22, 23]. Presentations of the underlying multi-level matrix groups support several of these results [9, 26, 31, 39]. Minimal theories are also known for several fragments and for all quantum circuits [14, 21]. Most of the sixteen results that QCert checks are among these (Section 5).

Mechanized quantum compilers and program logics establish program correctness or the soundness of circuit transformations [17, 36, 48, 58]. The mechanized completeness results for Π and its square-root extension are coherence arguments [16, 18]. Human-supervised, AI-assisted workflows have also formalized quantum program logics, algorithms, and components of interactive proofs [40, 49, 57]. QCert focuses on completeness and minimality of circuit equational theories.

Orientation. In Section 2, we review the two main strategies for proving completeness of a rule set and how minimality is shown. In Section 3, we describe the QCert framework, which adds matrix semantics to the library of circuits and presentations of Bian and Feng [7], together with our formalization workflow, how QCert keeps completeness from being lost in translation, and a comparison of the Agda and Lean formalizations. In Section 4, we present the formalization of qudit Clifford completeness that QCert grew out of, and the mathematical choices it fixed. In Section 5, we use QCert to machine-check the completeness and minimality results of sixteen papers. In Section 6, we describe the issues these formalizations found in the source papers. In Section 7, we discuss the current limitations of QCert and our plans to extend it.

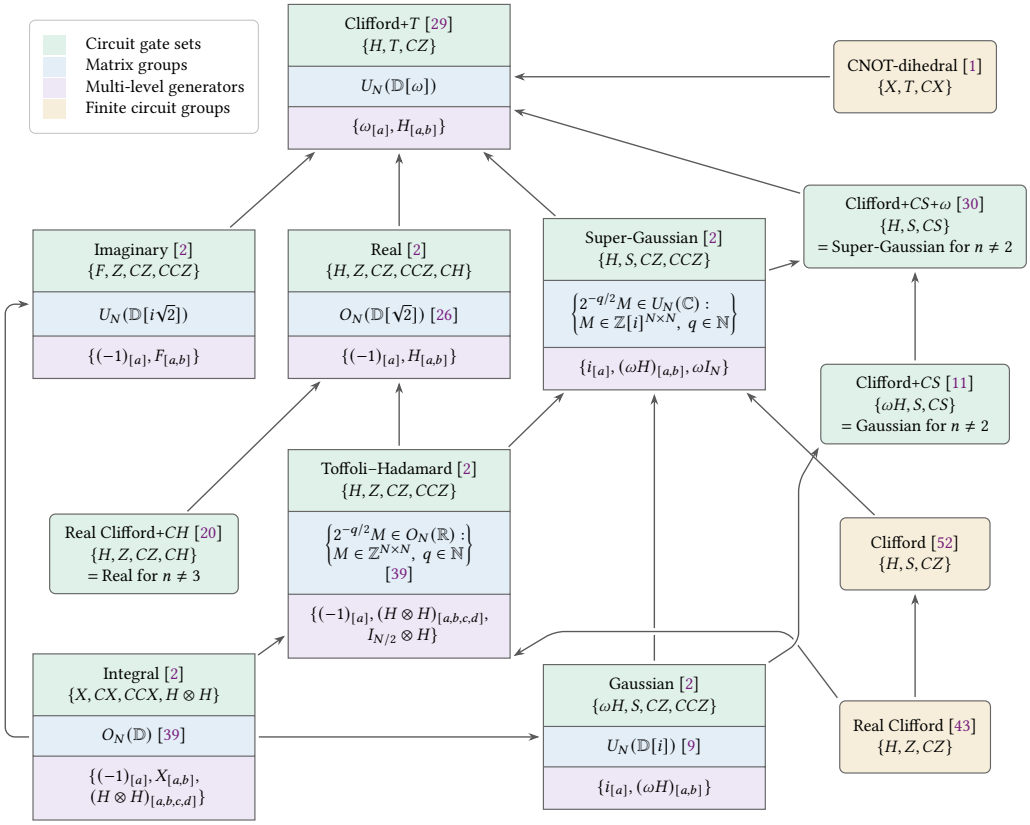


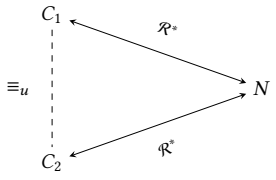
Fig. 2. Restricted Clifford+T circuit families and their underlying matrix groups, with $N = 2^n$ and $n \geq 1$. Each three-part box lists circuit gates, the group they represent with at most one clean ancilla, and reduced multi-level generators for that group. Without ancillas, the circuit group may be smaller. Arrows denote inclusion of ancilla-free circuit groups and, where shown, matrix groups. Mint-only boxes mark when related circuit groups agree; each exceptional inclusion is strict. Ivory boxes are finite circuit groups. Here $\mathbb{D} = \mathbb{Z}[1/2]$, $\omega = e^{i\pi/4}$, $F = (H + iI)/\sqrt{2}$, and q ranges over nonnegative integers. Subscripts specify distinct basis levels; $I_{N/2} \otimes H$ and ωI_N act globally. In both Gaussian cases, $X_{[a,b]} = (\omega H)_{[a,b]} i^2_{[b]} (\omega H)^\dagger_{[a,b]}$ is redundant. Clifford+CS+ω denotes adjoining the scalar ω to Clifford+CS.

2 Background

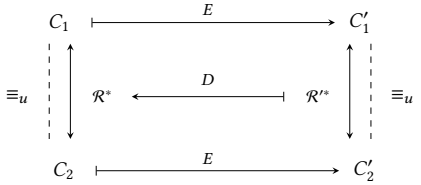
A circuit is a word over an alphabet $\mathcal{G}$ of gates. A rewrite rule is then a relation between words over this alphabet, and a rule set $\mathcal{R}$ generates a congruence on circuits. To a circuit C we associate an *interpretation* $[\![C]\!]$, which is its underlying matrix. The completeness results we formalize have matrix entries in the finitely generated rings $\mathbb{Z}\left[1/\sqrt{2}, i\right]$, $\mathbb{Z}\left[1/2, i\right]$, $\mathbb{Z}\left[1/\sqrt{2}\right]$, and $\mathbb{Z}\left[1/2\right]$, which make them exactly representable in Agda. We leave the formalization of universal circuits taking values in $\mathbb{C}$ for future work.

There are two main strategies for proving completeness of a rule set $\mathcal{R}$ over a gate set $\mathcal{G}$, both of which we support. Consider any two circuits C_1 and C_2 over $\mathcal{G}$ that have the same linear map, $[\![C_1]\!] = [\![C_2]\!]$. We need to show that C_1 can be rewritten into C_2 using $\mathcal{R}$. Figure 3a outlines the first strategy. It specifies a unique normal form for each linear map [1, 8, 13, 27, 28, 37, 38, 43, 50, 52],

often by giving an exact synthesis algorithm [9, 19, 26, 31, 39]. Completeness follows by proving that every circuit can be transformed into its normal form using the rewrite rules. By uniqueness, C_1 and C_2 have the same normal form N. Reversing the path from C_2 to N and concatenating it with the path from C_1 to N then gives the desired path of rewrites.



(a) The rewrites from C_1 and C_2 to their common normal form N using $\mathcal{R}$ give rewrites from C_1 to C_2 using $\mathcal{R}$.



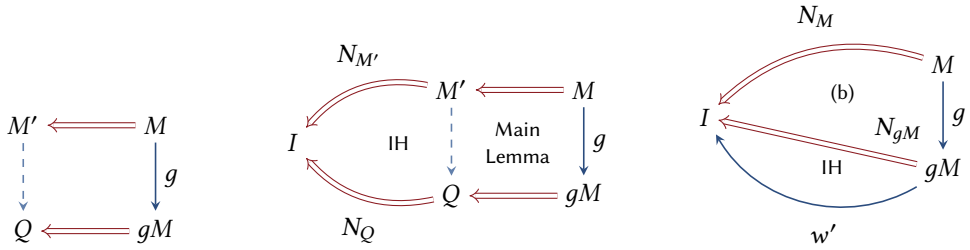
(b) Completeness of $\mathcal{R}'$ gives rewrites between $C_1' = E(C_1)$ and $C_2' = E(C_2)$. Translating this proof back gives rewrites between C_1 and C_2 using $\mathcal{R}$.

Fig. 3. Two strategies for proving completeness of $\mathcal{R}$ over a gate set $\mathcal{G}$. $\overset{\mathcal{R}^*}{\longleftrightarrow}$ means applying a finite sequence of rewrites using $\mathcal{R}$ in either direction, where $*$ denotes zero or more steps.

The exact synthesis results above present groups of matrices rather than circuits (the first block of Table 1). Their generators are *multi-level* matrices. For an $m \times m$ matrix M and distinct indices $a_1, \ldots, a_m$ in $\{0, \ldots, n-1\}$, the *m-level* matrix $M_{[a_1,\ldots,a_m]}$ is the $n \times n$ matrix that acts as M on the standard basis vectors $e_{a_1}, \ldots, e_{a_m}$ and as the identity on the others. For example, $(-1)_{[a]}$ negates e_a, and $\mathrm{X}_{[a,b]}$ exchanges e_a and e_b. Circuits on k qubits have matrices of dimension $n = 2^k$, but these groups are defined in every dimension n. We introduce multi-level matrices for two reasons. The gap we find in Section 6 is in a presentation by one- and two-level matrices [26], and our new result of n-qubit Toffoli-Hadamard completeness builds on a presentation by one-, two-, and four-level matrices [39].

For these groups, the normal forms come from exact synthesis, and the proof that every circuit can be transformed into its normal form has the same structure for each algorithm. Each matrix, or *state*, has a *level*, built from quantities such as the least denominator exponent of a column, and the algorithm reduces each state M to the identity in steps that lower the level. These steps form the normal word N_M. The *Main Lemma* says that for every state M and generator g, a diagram of relations closes the square formed by g and the algorithm's next steps from M and gM, passing only through states of lower level (Figure 4a). By well-founded induction on the level, the relations then rewrite $N_{gM}\, g$ into N_M (Figure 4b), and by induction on length, every word that reduces M to the identity rewrites into N_M (Figure 4c). Two words with the same matrix therefore rewrite into each other. These inductions have the same form for every presentation, so the main work in each formalization is its Main Lemma.

Figure 3b illustrates the second strategy, which uses an already complete rule set $\mathcal{R}'$ over a different gate set $\mathcal{G}'$ [3, 10–12, 14, 20, 22, 23, 25, 51]. Here, E and D translate circuits from $\mathcal{G}$ to $\mathcal{G}'$ and back, replacing each gate with a circuit over the other gate set. Both E and D preserve circuit composition and the underlying linear map. To transfer completeness from $\mathcal{R}'$ to $\mathcal{R}$, we first show that every rule of $\mathcal{R}'$, translated by D, can be derived using $\mathcal{R}$. We can then replace each step of the proof between C_1' and C_2' by its translated derivation, giving $D(C_1') \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_2')$. It then remains to prove that the rewrite $C_j \overset{\mathcal{R}^*}{\longleftrightarrow} D(C_j')$ is derivable, for which it suffices to prove the round-trip rewrites $g \overset{\mathcal{R}^*}{\longleftrightarrow} D(E(g))$ for each $g \in \mathcal{G}$. Sometimes the two strategies are combined, as in the scaled dyadic extension of [39].



(a) The Main Lemma. (b) Induction on the level. (c) Induction on length.

Fig. 4. Completeness from the Main Lemma. Plain arrows are words, double arrows are steps or runs of the algorithm, and dashed arrows are words through states of lower level.

Starting from a complete rule set, we can remove a redundant rule by deriving it from the other rules, thereby preserving completeness. To show that the resulting rule set is minimal, we need to give, for each remaining rule R, an interpretation of circuits that every other rule preserves but R does not. Any derivation using the other rules preserves this interpretation, so R cannot be derived from them [14, 21].

3 The QCert Framework

Our Agda framework QCert states every completeness theorem as a group presentation. Unlike the letters of an ordinary word, gates act on wires. The gates in $\mathcal{G}$ are indexed by the number of wires they act on. Wires are numbered from 0 at the bottom, and a gate always acts on the bottom wires of a circuit. To place it anywhere else, we apply a shift, which takes a circuit on n wires and places it on wires $1, \ldots, n$ of a circuit with $n + 1$ wires. Each relation is stated on as many wires as the paper draws it on. To apply it inside a circuit with more wires, we need *structural rules*, which hold for every circuit, regardless of its gate set. These rules say that an equation can be shifted up by one wire, gates on disjoint wires commute, and a scalar can be placed on any wire. These define the equations of a PROP [41], and are often defined implicitly in completeness papers, but a formalization in Agda must define them explicitly.

We associate to circuits a matrix semantics $[\![-]\!]$ with entries in a finitely generated ring like $\mathbb{Z}\left[1/\sqrt{2}, i\right]$. Hence, $[\![-]\!]$ defines a group homomorphism to a unitary matrix group G. We build G from the group structures of Agda's standard library, which is checked only once per ring. As matrices, a gate on the bottom wires and a shift are Kronecker products with an identity, so the extension of $[\![-]\!]$ from gates to circuits is defined once for all gate sets. Since the structural rules are universal, their soundness is also proved only once. Then for each paper, it suffices to check the soundness of its own rules. Soundness and completeness say the relations present the image of $[\![-]\!]$, which is the subgroup of G generated by the gates. In Agda, mathematical structures such as groups are defined as records. The record $\mathcal{R}$ IsSubPresentationOf G contains the semantics $[\![-]\!]$, with proofs of soundness and completeness. The record $\mathcal{R}$ IsPresentationOf G adds surjectivity, so the relations present all of G.

Figure 5 summarizes the two obligations: checking each rule's matrix equality for soundness, and deriving a known complete rule set $\mathcal{T}$ from $\mathcal{R}$ for completeness.

Each formalization ends with its main theorem stated as an IsSubPresentationOf or IsPresentationOf record, in one format for all of them. The relations are written n CRel,_===_ for circuits on n wires and n MRel,_===_ for words over multi-level matrices of dimension n. For example, Statement (1) is the theorem for the real stabilizer circuits of [43], with the relations R1–R16 of its Figure 1

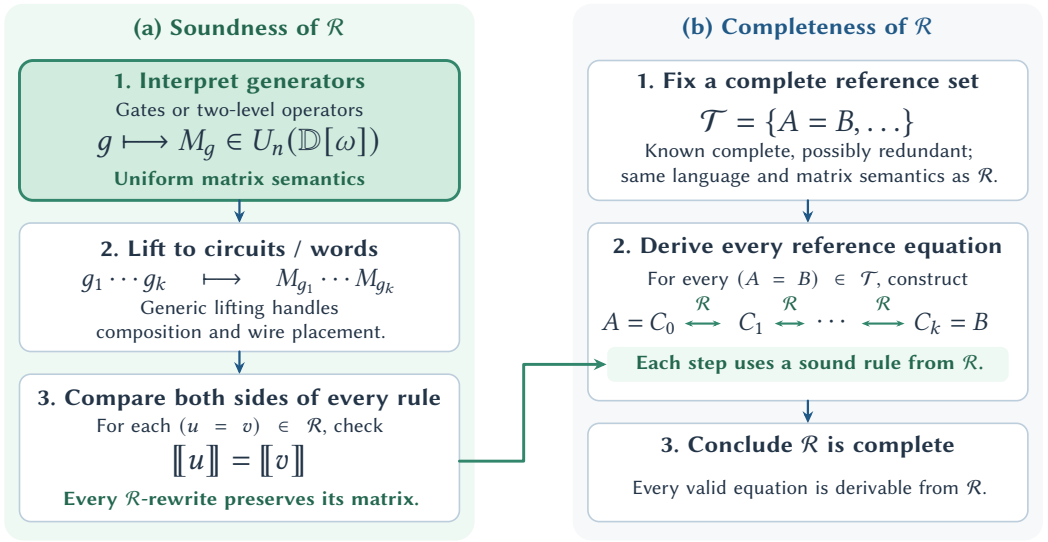


Fig. 5. QCert's shared matrix interpretation supports both proof obligations. The left column checks every rule of $\mathcal{R}$; the right derives every rule of a known complete set $\mathcal{T}$ in the same circuit language and matrix semantics. Each rewrite step uses a rule of $\mathcal{R}$, whose soundness is checked on the left. The displayed $U_n(\mathbb{D}[\omega])$ covers qubit fragments [2], with $\mathbb{D} = \mathbb{Z}[1/2]$ and $\omega = e^{i\pi/4}$. Qudit circuit families use other rings.

lifted to n wires. Statement (2) is the theorem for the two-level matrices of Hadamard-Π [26], with the relations of its Figure 6. Completeness and soundness are read off the record in one form too, Statements (3) and (4), where `⊢ w ≈ v` says that w and v are equal by the relations. The qubit developments compare matrices over $\mathbb{D}[\omega]$ (`_≈ᵁ_`), in which each reads its group; the qudit ones compare in their own groups.

```
presentation : ∀ n → (n CRel,_===_) IsSubPresentationOf U (2 ^ n)      (1)
presentation : ∀ n → (n MRel,_===_) IsPresentationOf (O n)             (2)
complete : ∀ {n} {w v : Circuit n} → [[ w ]] ≈ᵁ [[ v ]] → ⊢ w ≈ v      (3)
sound : ∀ {n} {w v : Circuit n} → ⊢ w ≈ v → [[ w ]] ≈ᵁ [[ v ]]         (4)
```

The minimality results of [14] take the same form, Statement (5), for the rule set of a fragment on n wires, `n ⊢_===_`. It is complete if it derives every equation that the matrices validate, and minimal if, without any one of its rules on n wires, it is no longer complete there. Minimality only makes sense for a rule set, which says what dropping a rule is, so Agda accepts `IsMinimal` only for `n ⊢_===_`.

```
minimal-complete : ∀ {n} → (n ⊢_===_) IsMinimal × (n ⊢_===_) IsComplete   (5)
```

Agda checks every proof, so a reader only needs to check that each theorem statement matches its paper's result. Since every theorem is one of these two records, it suffices for a reader to check only four components: the relations transcribed from the paper, the target group, the matrix of each gate, and the line assigning $[\![-]\!]$ to the record. Each Agda development collects these components and its main theorem in one module, `MainTheorem`, and the file `MainTheorems.agda` imports one such module per source and restates each theorem; the corresponding Lean statements are collected in `QCert/Table1.lean`.

Extensions of the earlier framework. To support these formalizations, QCert extends the Agda framework of Bian and Feng [7]. Besides the matrix semantics above and the statement of minimality (Section 2), it adds three features.

- *Three-wire gates.* The earlier framework allows gates on one and two wires. QCert also allows gates on three wires, such as the Toffoli gate, together with the structural rule they need.
- *Scalars.* QCert also allows scalars, which act on no wire, such as the global phase ω of Clifford circuits, with a structural rule that places a scalar on any wire. A rule set can then relate circuits exactly rather than up to a global phase.
- *PROPs.* In the earlier framework, a swap is an ordinary gate whose laws must be listed among the rules. QCert can also treat circuits as a PROP. One two-wire gate is designated as the swap, and the framework supplies its laws, that it is an involution and that every gate can be moved across it, once for all gate sets. A paper's rule set then lists only its own rules, each on the wires it is drawn on, while the swap and its laws stay hidden. We use this for the circuits of [1, 13, 14].

Where the examples of Bian and Feng [7] are groups such as symmetric, cyclic and Pauli groups, QCert formalizes sixteen published completeness and minimality results (Table 1).

3.1 Formalization Workflow

Our workflow has three parts. First, before formalizing a paper, we check its rewrite rules, its intermediate results, and each line of its derivations with exact matrices in short scripts. This caught most typos before any Agda code was written. Second, we state each main theorem in QCert before a coding agent starts, so the agent only supplies the proof. Finally, for long derivations, the agent writes a script that turns each line of the paper into one step of the Agda proof, so a failing step points to a specific line of the paper.

The main computational cost came from Agda's conversion checker. If two terms are syntactically identical (i.e., written in the same way), it accepts them immediately. Otherwise, it evaluates both, which can take exponential time for a long matrix product. For such products, we keep circuit interpretations abstract and decide matrix equality by computing a single Boolean using row operations, which is orders of magnitude faster than comparing matrices entry by entry. We record these techniques in the instructions followed by the agents, so later proofs avoid the same slowdowns.

3.2 Avoiding Completeness Lost in Translation

A machine-checked proof establishes only what its statement says. As the Navier–Stokes example in the introduction shows, a formalization can end up verifying a weaker result than the one intended [6]. For a completeness result, the meaning can be lost when the rules, the semantics, or the theorem is translated into a proof assistant. QCert addresses this in three ways.

Foundations. QCert builds on our formalization of qudit Clifford completeness and keeps the two mathematical choices that Section 4 explains. The semantics is constructed independently of the circuit syntax. The target group is defined by its own mathematics, as a group of matrices or a semidirect product, and not as a quotient of circuits, so completeness relates two independently given objects and cannot hold by construction. Each formalization also states what it does and does not verify. LLMs help construct the proofs and the proof assistant checks them, but we remain responsible for the definitions, the proof strategy, and the connection to the intended mathematics.

Explicit proof obligations. A proof by the second strategy of Section 2 starts from the sound rules $\mathcal{R}$ of the source paper and uses them to derive the translation of every rule of a rule set known to be complete (Figure 3b). This strategy suits QCert, because it turns completeness into explicit proof obligations, one for each rule of the complete set, which Agda either accepts or rejects. When an obligation cannot be discharged, say a rule r does not follow from $\mathcal{R}$, there are two possibilities. Either a derivation or a case is missing, and we supply it, or r must be added to $\mathcal{R}$. In the second case, the completeness claim concerns the revised rule set, and the formal statement says so. The relation r_{K} of Section 6 is an example. It does not follow from the rules of [3], so we added it and proved that it is independent of the others.

Shared structure. The matrix groups of different circuit families are mathematically related, often as subgroups of one another. The first five rows of Table 1 present five of these groups, and several circuit results are proved by translating into them. For example, two-qubit Clifford+T circuits are translated into $U_4\left(\mathbb{Z}\left[1/\sqrt{2}, i\right]\right)$, three-qubit Clifford+CS circuits into $U_8\left(\mathbb{Z}\left[1/2, i\right]\right)$, and real Clifford+CH circuits into $O_{2^n}\left(\mathbb{Z}\left[1/\sqrt{2}\right]\right)$. A common framework reflects this structure rather than merely collecting separate proofs. We verify general arguments and algebraic constructions once, such as Reidemeister–Schreier coset enumeration and presentations of semidirect products, and reuse them by checking only the assumptions and semantic connections specific to each family. This reduces the need to translate each proof independently and makes the dependencies of each completeness claim explicit.

3.3 Comparison of the Agda and Lean Formalizations

The Lean [24] version of QCert formalizes the same sixteen results on top of Mathlib [54]. Once the Agda formalization exists, translating it to Lean is comparatively easy, so the Lean version is not a second achievement of the same kind. Its point is its theorem statements. They rest on Mathlib's standard definitions, with matrix groups over Mathlib's real or complex numbers in place of our exact rings, so a reader can check them against the source papers using definitions that are already widely used and reviewed. For qudit Clifford, the Lean statement also removes an argument that the Agda statement leaves to external mathematics (Section 4.4).

Nor is the Lean framework an independent implementation. We instructed the coding agent that wrote it not to consult our Agda implementation, but the Lean framework is nonetheless a translation of the Agda one. Its words, circuit generators, structural rules, and presentation and sub-presentation predicates are those of the Agda framework, down to the names of the definitions and the wording of their documentation. The two formalizations therefore share the design of their framework, and would share any mistake in it, so their agreement is weaker evidence than two independent formalizations would give. They use different representations and sometimes different proofs, so the theorem statements of each must be checked against the source papers separately. Below we compare their definitions, proof development, verification time, and execution time (Figure 6). Table 2 in Appendix A summarizes the comparison. These observations concern our two formalizations, not Agda and Lean in general.

Definitions and semantics. The translation keeps the definitions but adapts their representation to Mathlib. Agda uses its own word type, with explicit brackets, and the setoid-based algebra of the standard library. A presentation is a record containing a semantics and a group isomorphism, or a monomorphism for a sub-presentation. Lean uses Mathlib's free monoid and the monoid congruence generated by the relations. Its presentations are predicates in `Prop`, from which it

derives isomorphisms out of the quotient. In both, the circuit infrastructure, the structural rules, and general completeness arguments are shared between developments.

The semantic targets also differ. Most Agda developments work directly with exact representations of matrix entries. Lean states its matrix groups over Mathlib's real or complex numbers and proves bridges to them from computable exact rings. In particular, the Agda qudit Clifford development presents the constructed model of Section 4, whereas Lean proves soundness and completeness against complex matrices for every odd prime. A common source therefore does not imply identical statements or intermediate lemmas.

Proof development. Excluding blank lines and comments, the Agda formalization has about 355,000 lines and the Lean formalization about 339,000 (Figure 6(a)). Lean's is shorter in thirteen of the sixteen rows, by a factor of five or more for qudit Clifford and qutrit Clifford. It is longer for Hadamard-Π, three-qubit Clifford+CS, and real Clifford+CH. For Toffoli–Hadamard we count the Agda formalization that, like the Lean one, reaches the result through the two-level presentation of [39]. The repository has a second one, which replays the authors' own derivation traces in about 28,600 lines, four fifths of them generated certificates. Both counts include the derivations and certificates that scripts generate. The Agda proofs are explicit terms, built by pattern matching and equational reasoning, with reflection-based solvers for associativity and ring identities. Its library develops the group theory the proofs need, in about 13,200 lines against 1,200 in Lean. The Lean proofs rely on Mathlib's algebra and on tactics, with about 69,000 calls to automation such as `simp`, `omega`, `decide`, and `ring`.

Both implementations reuse normal-form, exact-synthesis, and translation arguments. Both also check generated derivations with a replay procedure that is proved correct. The proof assistant checks the computation and the theorem that a successful replay gives a derivation, so the scripts that search for derivations need not be trusted. For Clifford+CS, Lean constructs its own coset certificates. The trust checks reflect the different foundations. Every Agda module is `--safe` and uses no postulates, so the Agda corpus is constructive. The Lean corpus uses no `native_decide`, and its main theorems depend on no unfinished proof. An audit of 105 declarations reports only the axioms `propext`, `Classical.choice`, and `Quot.sound`.

Verification time. Table 1 and Figure 6(b) give the check times, both measured on the same machine. In total Agda took 2,399 seconds and Lean 9,305, or 1,979 and 3,826 without real Clifford+CH, whose Lean build alone took 5,479 seconds, more than half of the total. Lean took between 1.2 and 4.8 times as long as Agda on twelve rows, and 13 times as long on real Clifford+CH. Its five-wire certificate modules `Family267` and `Family268` alone took about 1,741 and 1,369 seconds. Lean was faster for three rows, Clément's presentation, qudit Clifford, and qutrit Clifford, whose Lean proofs take different routes. Both assistants decide equalities of matrices and certificates by evaluation in the checker. Agda reduces a Boolean to `true`, and Lean uses `decide +kernel`.

Both columns were measured under WSL on an Intel Core i5-13400TEF with sixteen logical CPUs, about 26 GiB of RAM, and 64 GiB of swap. The Agda column comes from a check from scratch of revision `88440624` on 8 November 2026, with Agda 2.8.0 and version 2.4 of its standard library. We removed the repository's interface files, kept those of the standard library and EucDomain, and checked the shared library and then each row in an Agda process of its own, through a module that imports every module of the row. Agda's profiler reports the time spent on each module without its imports, and a row's time is the sum over its own modules, wherever they were checked. It leaves out process startup and the loading of interface files: the whole check, shared library included, took 3,210 seconds of wall time, and no process used more than 5.1 GiB. No other proof assistant ran during the check. The Lean column comes from a clean build of revision `e19af70c`

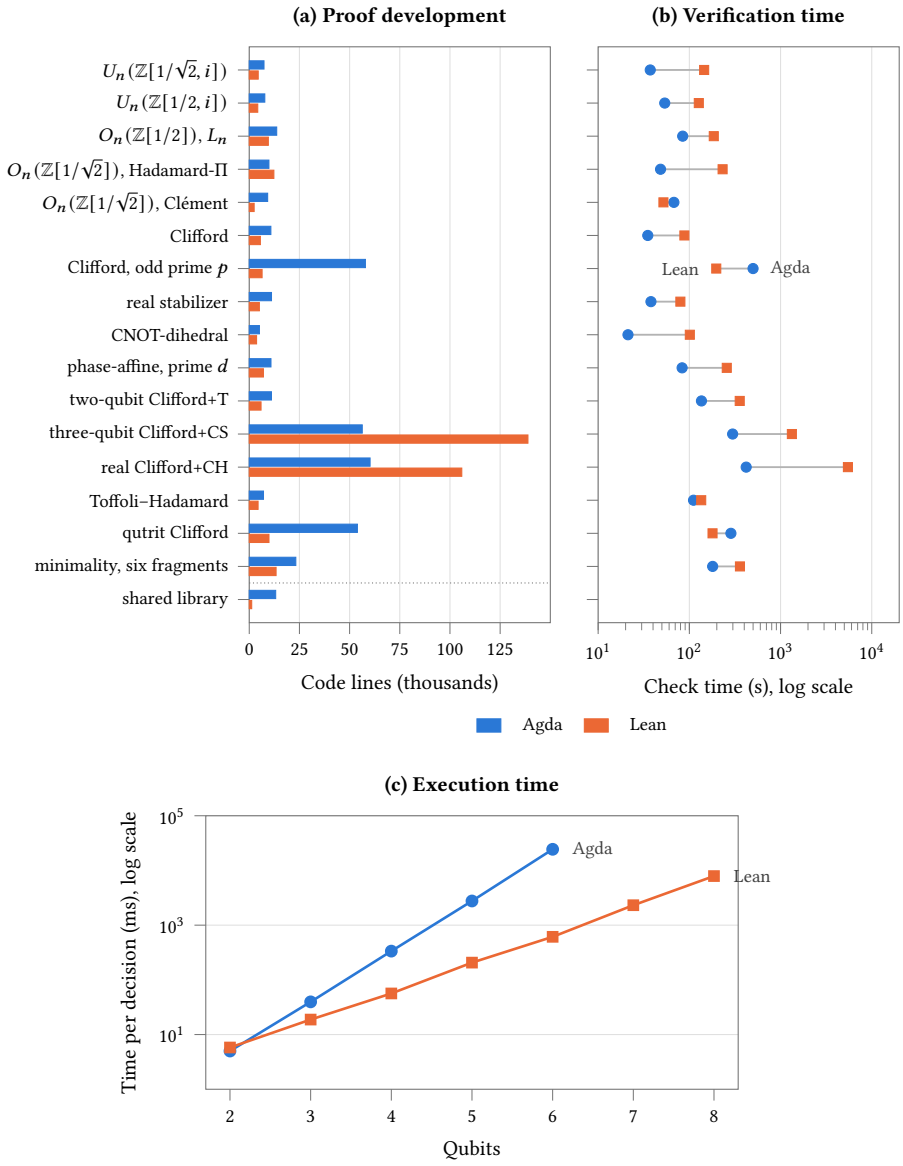


Fig. 6. The Agda and Lean formalizations of each row of Table 1. (a) Code lines, excluding blank lines and comments. The last row is the shared library of each framework. (b) The check times of Table 1, Agda (circles) and Lean (squares), measured on the same machine as described in the text. (c) The time each compiled formalization takes to decide that two Clifford circuits of 100 gates have the same matrix, by the procedure its soundness proof uses, as medians over three runs of five random circuits.

on 5–6 November 2026, with Lean 4.30.0 and the pinned Mathlib revision. We removed QCert's build artifacts, kept the Mathlib and other dependency caches, and built the folders one at a time in dependency order with `LEAN_NUM_THREADS=2`. Each time is the elapsed wall time of a folder, including Lake startup and artifact generation, and a shared dependency is counted only in the

folder that defines it. The two columns therefore measure different things, per-module checking time in Agda and folder build time in Lean. Proof constructions, module boundaries, parallelism, and artifact generation also differ, so the differences cannot be attributed to the proof assistants alone. We have yet to separate the time Lean spends running automation from the time it spends checking the resulting proofs.

The build covered all 810 tracked library modules, excluding three generator templates, and took **9,415.37 seconds (2 h 36 min 55 s)** in timed build commands. Of these modules, 808 passed. Two unused legacy helpers in real Clifford+CH failed, `ItemBFun` at `omega` proofs and `ItemASteps` at the heartbeat limit. Neither is imported by the main theorem chain or the audit, both of which passed, and the real Clifford+CH entry of Table 1 includes the whole attempted build. Beyond the sixteen development folders, the shared `Circuit`, `Exact`, `Matrix`, and `NumberTheory` folders took 17.91, 15.51, 6.88, and 15.29 seconds, and the five root modules, including the audit, took 54.95 seconds. Lean builds of another project ran on the same machine during part of the run. We therefore re-timed the twenty folders and root modules they overlapped and report the second timing, during which the other project used about a quarter of one CPU on average. The two runs total 9,604 and 9,415 seconds, and the two timings of each development folder agree within 15%, except for Toffoli–Hadamard (178 and 135 seconds).

Execution time. The Agda normal forms are executable functions. The Lean developments obtain normal forms either as existence theorems or as `noncomputable` definitions. For example, the Lean proofs for the Clifford and real stabilizer circuits of [43, 52] show that every circuit equals a normal form without defining a function that computes it, and the exact synthesis of the two-level groups is noncomputable. In all, 460 definitions in the sixteen Lean developments are declared `noncomputable`, so there is no Lean normalizer to time. Both formalizations can, however, execute the procedure that decides whether two circuits have the same matrix, which their soundness proofs run. We compiled this procedure for the Clifford circuits of [52] to native code, through Agda's GHC backend and Lean's C backend. We then timed both on an Intel Core i5-14400, on the same random circuits of 100 gates over H, S, and CZ, deciding that each circuit w has the matrix of $w\,S^4$ (Figure 6(c)). On two qubits Agda is slightly faster, at 5.0 against 5.8 milliseconds per decision. From three qubits on Lean is faster, by a factor of 2.1 on three qubits and 40 on six, where Agda takes 24 seconds and Lean 0.61. The gap comes mostly from the procedures rather than the compilers. Agda computes each circuit's matrix as a product of stored $2^n \times 2^n$ matrices, so its work grows about eightfold with each qubit. Lean simulates the circuit on each basis vector, so its work grows about fourfold.

4 Case Study: Qudit Clifford Completeness

QCert grew out of our formalization of the completeness theorem of Bian, Li et al. [8]. That theorem gives sixteen rewrite rules that are sound and complete for multi-qudit Clifford circuits over H, S, and CZ in every odd prime dimension p. Unlike the other fifteen formalizations, this one was built largely by hand, and it fixed the mathematical choices that the rest of QCert follows. It is checked in safe, axiom-free Agda against a semantics constructed independently of the syntax. The proof follows the paper's except at one step, where we instead *factor the group, present the factors, and compose the presentations* (Section 4.1).

4.1 Qudit Clifford Circuits and the Main Theorems

We use the gate conventions of Bian, Li et al. [8]. Fix an odd prime p and let $\omega = e^{2\pi i/p}$. On the basis $|0\rangle, \ldots, |p-1\rangle$ indexed by $\mathbb{Z}_p$, the gates $X|j\rangle = |j+1\rangle$ and $Z|j\rangle = \omega^j|j\rangle$ generate, with the

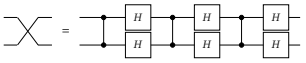
phases ω^t, the Pauli group $\mathcal{P}_n$ on n qudits. The gates

$$H\colon |j\rangle \mapsto \frac{\delta_p}{\sqrt{p}} \sum_k \omega^{jk} |k\rangle, \qquad S\colon |j\rangle \mapsto \omega^{j(j-1)/2} |j\rangle, \qquad CZ\colon |j\rangle|l\rangle \mapsto \omega^{jl} |j\rangle|l\rangle$$

generate, with the same phases, the Clifford group $\mathcal{C}_n$. Here $\delta_p = (-i)^{(p-1)/2}$, which makes the determinant of H equal to 1. Modulo scalars, we write $\overline{\mathcal{C}}_n = \mathcal{C}_n/\langle\omega\rangle$ and $\overline{\mathcal{P}}_n = \mathcal{P}_n/\langle\omega\rangle \cong \mathbb{Z}_p^{2n}$. Then $\mathcal{C}_n/\mathcal{P}_n \cong \overline{\mathcal{C}}_n/\overline{\mathcal{P}}_n \cong \mathrm{Sp}(2n, \mathbb{Z}_p)$ and $\overline{\mathcal{C}}_n \cong \overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. We do not formalize these last two isomorphisms, because our semantic groups are *defined* as their right-hand sides. Bian, Li et al. [8] also include -1 among the global phases, so that the swap and the multiplier gates belong to $\mathcal{C}_n$. We work mainly in the projective quotient, where -1 makes no difference and the relations are slightly simpler.

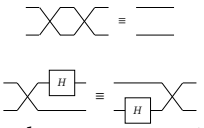
Using the framework of Section 3, we encode circuits, rules, and reasoning with the rules. We draw circuits in matrix-multiplication order, the reverse of the usual circuit order, so $w \bullet v$ applies v first but draws it on the right. For example, this is the swap:

```
Ex : Word (Gen (2+ n))
Ex = CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑
```



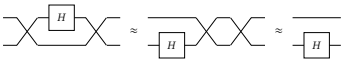
A rule set is an inductive family indexed by the width, with one constructor per rule. Each rule is stated once, on the bottom wires of every width at which it fits. For example, these are two of the fifteen constructors of our primary rule set _SRel,_===_, the order of the swap and the rule moving H across it:

```
data _SRel,_===_ : (n : ℕ) → WRel (Gen n) where
  order-Ex     : (2+ n) SRel, Ex ^ 2 === ε
  semi-Ex-H↑ : (2+ n) SRel, Ex • H ↑ === H ↓ • Ex
```



Reasoning with the rules is equational reasoning in the congruence $\approx$ they generate, with combinators for symmetry, associativity, and rewriting the left or right factor of a product. For example, conjugating by the swap moves H down a wire:

```
conj-Ex-H↑ : Ex • H ↑ • Ex ≈ H ↓
conj-Ex-H↑ = begin
  Ex • H ↑ • Ex   ≈⟨ sym assoc ⟩
  (Ex • H ↑) • Ex ≈⟨ cleft axiom semi-Ex-H↑ ⟩
  (H ↓ • Ex) • Ex ≈⟨ assoc ⟩
  H ↓ • Ex • Ex   ≈⟨ cright axiom order-Ex ⟩
  H ↓ • ε         ≈⟨ right-unit ⟩
  H ↓ ∎
```



We can then state our main theorems:

Theorem 4.1 (Paper-V1.presentation). *For every width n, the fifteen rules of* `Paper-V1` *(Section 4.3) present the projective Clifford group:*

```
presentation : ∀ n → (PapV1.Clifford-Relations._QRel,_===_ n) IsPresentationOf (Pauli⋊Sp n)
```

These rules are the relations of Bian, Li et al. [8] read modulo scalars, with its rule (C4) in an equivalent form. The relation _QRel,_===_ closes _SRel,_===_ under the structural rules (Section 3), and Pauli⋊Sp n is the semidirect product of $\mathbb{Z}_p^{2n}$ by $\mathrm{Sp}(2n, \mathbb{Z}_p)$ (Section 4.3). The record _IsPresentationOf_ (Section 3) gives a group isomorphism from circuits modulo $\approx$ to $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n, \mathbb{Z}_p)$. That this map is well defined is *soundness*, since each rule holds. That it is injective is *completeness*, and that it is surjective says that every projective Clifford operator is represented by a circuit.

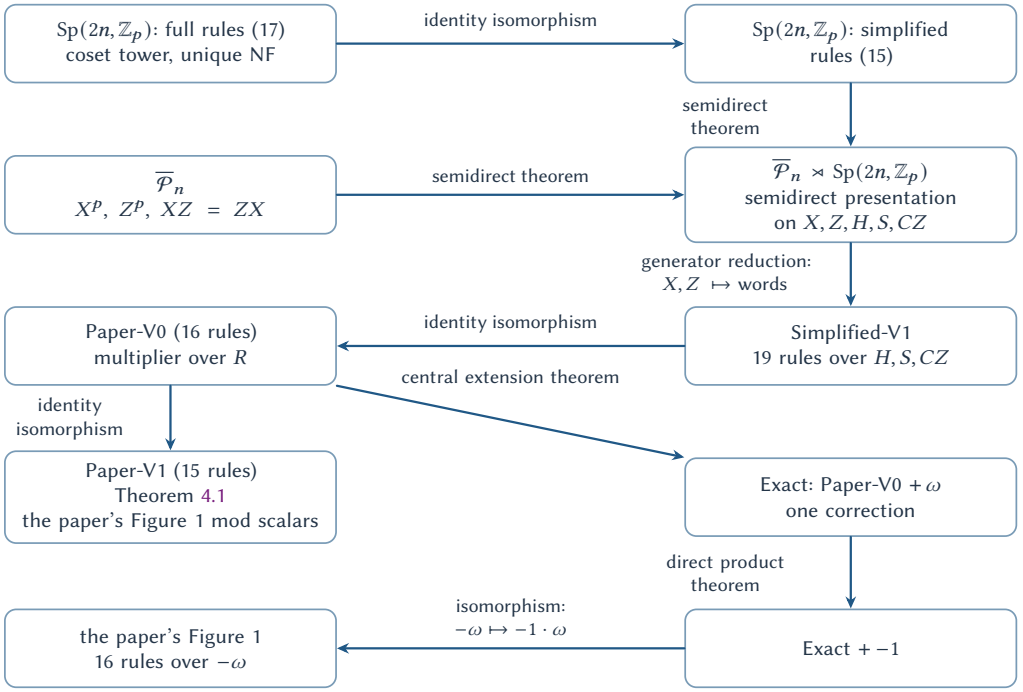


Fig. 7. The chain of presentations used to formalize the qudit Clifford result of Bian, Li et al. [8]. Every arrow is a verified presentation theorem or isomorphism of presentations. The symplectic stages present $\mathrm{Sp}(2n,\mathbb{Z}_p)$, and the projective chain from the semidirect presentation to Paper-V1 presents $\overline{\mathcal{P}}_n \rtimes \mathrm{Sp}(2n,\mathbb{Z}_p)$. "Exact" is a central extension by $\langle\omega\rangle$, and the final two stages also include the direct factor $\langle -1\rangle$.

We refine the normal form of Bian, Li et al. [8] for symplectic operators, which are Clifford operators modulo Paulis, into a doubly inductive definition (Section 4.2), and we prove that normal forms are unique:

Theorem 4.2 (unique-nf). *Normal forms with equal symplectic denotations are equal:*

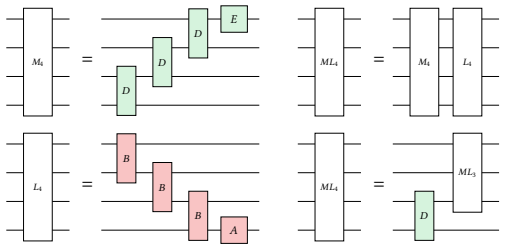
unique-nf : $\forall\ n\ \{u\ v : NF\ n\} \to [\![\ [\ u\ ]\ ]\!] \approx^s [\![\ [\ v\ ]\ ]\!] \to u \equiv v$

The route. We mostly follow the paper. Its normal boxes, its symplectic normal form, and its parametrized *box relations* reappear below. A box relation says that pushing a gate through a normal box yields an updated box and residual *dirty* gates. We also keep the reduction to eighteen relations of its Theorem 4.4, a step the paper already delegated to Agda. The one major divergence is the step that promotes symplectic to projective completeness, its Proposition 4.9. There, the paper corrects each rule $C_l = C_r$ to $C_l = C_r\,;P$ for the unique Pauli P that makes it projectively sound. It then replays the symplectic rewrites, pushing each correction to the end, where it becomes "a potentially different Pauli". We did not see how to verify this argument at reasonable cost. Instead, we work in a syntax where Paulis are trivial, since symplectic circuits have only H, S, and CZ and denote $\mathrm{Sp}(2n,\mathbb{Z}_p)$ itself. We then reintroduce Paulis and scalars by construction (Figure 7). The Pauli bookkeeping of Proposition 4.9 is thus done once, by a theorem about semidirect products that the framework proves for arbitrary groups.

4.2 The Symplectic Factor

We model $\mathrm{Sp}(2n, \mathbb{Z}_p)$ as the group of $\mathbb{Z}_p$-linear bijections of the phase space Pauli n = Vec $(\mathbb{Z}_p \times \mathbb{Z}_p)$ n that preserve the symplectic form. An element of the phase space lists, for each wire, the exponents of $X^a Z^b$ up to phase. Against this semantics, completeness rests on the normal form, which we refine into a doubly inductive datatype:

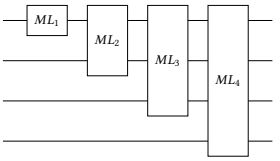
```
M L ML : ℕ → Set
M 0 = ⊤
M (1+ n) = Vec D n × E
L 0 = ⊤
L (1+ n) = Vec B n × A
ML 0 = ⊤
ML 1 = M 1 × L 1
ML m@(2+ n) = M m × L m
            ⊎ D × ML (1+ n)
```



An M-row is a column of D boxes ending in an E box, the paper's X-normal layer. An L-row is a column of B boxes ending in an A box, its Z-normal layer, with the A box at the bottom. An ML box at width $m \geq 2$ is either a full pair of rows or a bottom D box followed by an ML box one wire up, so it is a spine of D boxes capped by a pair of rows. This is the first induction, on the width of a single coset representative. The second induction is the staircase:

```
NF : ℕ → Set
NF 0 = ⊤
NF (1+ n) = NF n × ML (1+ n)

[_] : ∀ {n} → NF n → Word (Gen n)
[_] {0}     tt        = ε
[_] {1+ n} (nf , ml) = [ nf ] ↑ • [ ml ]ml
```



The table. The coset table is the paper's "push a gate through a normal box", written as a structurally recursive function ract with its soundness law ract-sound (about 5,600 lines together):

```
ract       : ML (1+ n) → Gen (1+ n) → Circuit n × ML (1+ n)
ract-sound : ∀ (c : ML (1+ n)) g → let (b' , c') = ract c g in [ c ]c • [ g ]w ≈ b' ↑ • [ c' ]c
```

Each clause of ract-sound is derived from the fifteen simplified symplectic relations. For instance, pushing S through an E box is one step on powers of S, namely $[\, b \,]^{\mathrm{e}} \bullet S = S^{-b} \bullet S \approx S^{-(b-1)} = [\, b - 1 \,]^{\mathrm{e}}$. A circuit is rewritten to its normal form by repeating this step gate by gate, so every rewrite is traced to a relation.

Uniqueness. The symplectic group acts on Vec Pauli_1 n, a $2n$-dimensional symplectic $\mathbb{Z}_p$-vector space in which Pauli_1 is the two-dimensional symplectic subspace of a single wire. The elements that fix the bottom wire's subspace pointwise form a copy of $\mathrm{Sp}(2(n-1), \mathbb{Z}_p)$ acting above the bottom wire, and the ML boxes represent its cosets. Every $f \in \mathrm{Sp}(2n, \mathbb{Z}_p)$ factors as $f = f' \circ c_n$, with $c_n \in ML_n$ and f' acting above the bottom wire, and c_n is unique because the image of the bottom wire determines the box. Recursing on f' writes f uniquely as $c_1 \circ c_2 \circ \cdots \circ c_n$ with $c_k \in ML_k$, which is Theorem 4.2.

Surjectivity. Every symplectic transformation is the action of some circuit. We prove this constructively by clearing the bottom wire and recursing on the rest. This is the paper's Lemma 3.7, with the coset box computed rather than asserted:

```
Theorem-LM : ∀ {n} (p q : Pauli n) → sform p q ≡ 1 → ∃ λ (lm : ML n) → act [ lm ]ml p ≡ pZ0 × act [ lm ]ml q ≡ pX0
```

Completeness. Circuits with the same denotation rewrite through the table to their normal forms. These normal forms denote the same map by soundness, so they are equal by Theorem 4.2, and the circuits are equivalent. This argument is the framework's by-normalization (Figure 3a). With surjectivity, it shows that the simplified relations, which their own module also calls _SRel,_===_, present the symplectic group:

```
presentation : ∀ n → (n SRel,_===_) IsPresentationOf (Sp-group n)
```

4.3 Composing: Paulis, the Semidirect Product, and Scalars

$\overline{\mathcal{P}}_n \cong (\mathbb{Z}_p \times \mathbb{Z}_p)^n$ is presented over X and Z on each wire by $X^p = Z^p = \varepsilon$ and $ZX = XZ$. Its semantic group is the phase space that $\mathrm{Sp}(2n, \mathbb{Z}_p)$ acts on, and its normal form is $X^a Z^b$ on each wire. Combining this with the symplectic presentation of Section 4.2 is an instance of the framework's theorem for semidirect products. The resulting presentation has the relations of both factors together with the conjugation relations $h\, n\, h^{-1} = conj\, h\, n$, where the action conj spells out the paper's Lemmas 2.24–2.25. The theorem asks that the action respect the relations of each factor, and we prove this semantically. The conjugates of congruent words have equal denotations by soundness, so they are congruent by Pauli *completeness*. For symplectic rules this is respects-Δ, and it corresponds to the paper's "Pauli complete" rules (its Definition 4.7). Since every premise is a theorem, the semidirect presentation of Pauli⋊Sp is unconditional.

Reaching the paper's rules. The semidirect presentation has generators X, Z, H, S, and CZ and $3 + 15 + 14$ relations (Pauli, symplectic, and conjugation), whereas the paper's rule sets have only H, S, and CZ. Three isomorphisms of presentations close this gap (Figure 7), through `Simplified-V1` (nineteen rules), `Paper-V0` (sixteen), and `Paper-V1` (the fifteen of Theorem 4.1). Along the way, X and Z become words, and the Pauli and conjugation rules become theorems.

The first isomorphism, from the semidirect presentation to `Simplified-V1`, is between different alphabets. Its map f sends S to $R = S \bullet Z^{1/2}$, and its inverse h sends S to $Z^{-1/2} \bullet S$. In particular, f induces the embedding of the symplectic group into the projective Clifford group. The nineteen rules of `Simplified-V1` are the fifteen simplified symplectic rules together with $(SH)^3 = \varepsilon$, $H^2SH^2S = SH^2SH^2$, and the two rules pushing X through CZ. From these, f-well-defined derives the images under f of the three Pauli rules and the fourteen conjugation rules.

The second isomorphism, from `Simplified-V1` to `Paper-V0`, is the identity on circuits. The two rule sets share six rules word for word and three more up to the spelling of the multiplier, and each derives the rest of the other. The larger direction is `Paper-V0`'s module `Lemmas`, of about 4,750 lines.

The third isomorphism, from `Paper-V0` to `Paper-V1`, changes how the multiplier $M_x : |j\rangle \mapsto |xj\rangle$ is spelled as a word. `Paper-V0` spells it over R, as $M_x = R^{x^{-1}} H R^x H R^{x^{-1}} H$. The paper's (T1) spells it over S, up to a phase, as $M_x = \mathrm{SHS'}\ x\ x^{-1}$, where $\mathrm{SHS'}\ a\ b = Z^{(b-1)/2} X^{(1-a)/2} S^b H S^a H S^b H$. Over S, the rule semi-MR reads $M_g S^{g^2} Z^{(g^2-g)/2} = S M_g$. `Paper-V1` restates the four multiplier rules over SHS' and drops one rule. Since M_1 is now the word $(SH)^3$, M-power at $k = 0$ already gives $(SH)^3 \approx \varepsilon$. Over R, M_1 is the different word $(RH)^3$, so that axiom had to stay. The bridge $R^a H R^b H R^a H \approx \mathrm{SHS'}\ b\ a$ for $ab = 1$ (`Shared.PauliBase`) carries the multiplier rules across, which gives Theorem 4.1. The paper's own rule (C4) states the conjugation of S by M_g in the other direction. Each of (C4) and semi-MR derives the other (SemiMS), so Figure 1 of Bian, Li et al. [8] read modulo scalars is isomorphic to `Paper-V1`.

Scalars. The theorem extension-presentation extends a projective presentation by a scalar. We apply it to `Paper-V0`, which has order-SH as an axiom. We add ω with $\omega^p = \varepsilon$, let ω commute with

every gate, and *correct* each rule by its scalar. The correction is ε except on order-SH, the one rule that does not hold exactly, since $(SH)^3 = \omega^{-1/8}$. The exponent $-1/8$ is computed in $\mathbb{Z}_p$, where it equals $(p^2-1)/8$. The multiplier words now denote $|j\rangle \mapsto |xj\rangle$ only up to a scalar.

The semantic extension is the Weyl cocycle on the natural semidirect product, $c((P,A),(Q,B)) = \frac{1}{2}\,\mathrm{sform}(P,\ A\cdot Q)$. Here $A\cdot Q$ is the symplectic part acting on the Pauli vector, and sform is the symplectic form of the phase space, the exponent in $PQ = \omega^{\mathrm{sform}(P,Q)}\,QP$. The cocycle identity follows from the bilinearity and the symplectic invariance of sform. The cocycle of the presented group is the pullback of c along the presentation by `Paper-V0`, so the two agree by construction. The identification with unitary operators uses Weyl multiplication [32, Eq. (4), Theorem 3] and a *linear* Weil representation with exact covariance [33, Theorem 1.3.2]. The operators $U(e,v,A) = \omega^e W(v)\rho(A)$ satisfy

$$U(e,v,A)\,U(f,w,B) = U\big(e+f+\tfrac{1}{2}\mathrm{sform}(v,Aw),\ v+Aw,\ AB\big).$$

This is the Heisenberg–Weil representation [34, §1.3]. Conjugation on Weyl operators determines A and v, so the representation is faithful and its scalars are exactly $\langle\omega I\rangle$ (at width zero, $e \mapsto \omega^e$). Adjoining $-I$ gives its direct product with $\mathbb{Z}_2$, whose scalars are $\langle -\omega I\rangle$. These finite phase choices differ from the full unitary normalizer, which contains $U(1)$. With no hypotheses, we obtain:

```
presentation-exact : ∀ n → (n Exact._===_) IsPresentationOf (PE.Presented-group n (exact-generator-data n))
```

Agda also shows that the trivial cocycle cannot realize the correction of order-SH (split-forces).

The rules as printed. Figure 1 of Bian, Li et al. [8] is stated over $-\omega$. Its multipliers carry Legendre symbols and its swap carries the sign $(-1)^{(p-1)/2}$, so it needs the -1 that the exact group lacks, since the scalars of that group have odd order. Adjoining -1 as a central involution presents the exact group times $\mathbb{Z}_2$, which is Clifford±. Figure 1 is isomorphic to this rule set by $-\omega \mapsto -1\cdot\omega$, where the scalar of each exact rule is determined by reading its lift back. This is the paper's Theorem 4.10, stated against our group:

```
presentation-Figure1 : ∀ n → (n F._===_) IsPresentationOf (Clifford± n)
```

4.4 Discussion

Everything above is checked by Agda 2.8 with standard library 2.4 under `--safe`, with no postulates and no holes. The development has 215 modules and about 58,000 lines of code, of which about 27,000 are for the symplectic group, 18,000 for the projective chain, and 13,000 for the scalars. These counts exclude the shared library (Section 3.3).

The Agda development does not formalize the two isomorphisms of Section 4.1 [32, 33], and no matrix appears in it. The representation argument above is external mathematics, and matching the paper's gate phases remains a convention check [8, 46]. The Lean development needs neither, since it proves soundness and completeness against complex matrices directly (Section 3.3).

We used versions of Claude for parts of this formalization: composing box relations to prove ract-sound, defining the symplectic group, and establishing the equivalence between the code's relations and those in Figure 1 of Bian, Li et al. [8], read modulo scalars. Claude also wrote presentation-exact and the layer for Figure 1 with -1.

5 Formalized Completeness and Minimality Proofs

The sixteen formalizations in Table 1 contain about 355,000 lines of Agda code and use no postulates, and each theorem is as general as the result in its source paper. Moreover, all matrices are computed exactly. Their entries lie in algebraic number rings, whose elements are given by finitely many integers, so Agda checks automatically whether two matrices are equal. Over the real or complex

Source	Object	Result	Agda Mod	Agda time (s)	Lean time (s)
Groups of m-level matrices					
Greylyn [2014]	$U_n(\mathbb{Z}[1/\sqrt{2}, i])$	presentation, $n \leq 4$	34	37.28	145.34
Bian and Selinger [2021]	$U_n(\mathbb{Z}[1/2, i])$	presentation	39	53.73	126.99
Li et al. [2021]	$O_n(\mathbb{Z}[1/2])$, L_n	presentations	71	84.52	185.32
Fang et al. [2026]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	46	48.33	231.57
Clément [2019]	$O_n(\mathbb{Z}[1/\sqrt{2}])$	presentation	43	67.72	51.99
Circuits					
Selinger [2015]	Clifford	sub-presentation	32	35.01	88.24
Bian, Li et al. [2026]	Clifford, odd prime p	presentation of a model	216	499.37	197.32
Makary et al. [2021]	real stabilizer	sub-presentation	31	37.94	79.75
Amy et al. [2018]	CNOT-dihedral	sub-presentation	31	21.24	101.22
Blake [2026b]	phase-affine, prime d	sub-presentations	57	83.30	257.24
Bian and Selinger [2023a]	two-qubit Clifford+T	sub-presentation in U_4	63	135.79	356.79
Bian and Selinger [2023b]	three-qubit Clifford+CS	sub-presentation in U_8	125	298.29	1330.89
Clément [2026]	real Clifford+CH	sub-presentation	348	419.99	5478.79*
Amy et al. [2024]	three-qubit Toffoli–Hadamard	presentation of L_8	37	111.17	134.91
Li et al. [2025]	qutrit Clifford	presentation	90	285.58	179.60
Minimality					
Blake [2026c]	six fragments	minimal, complete	80	180.10	358.87
Total			1,343	2,399.36	9,304.83

Table 1. Completeness and minimality results formalized in QCert. A presentation gives generators and relations for a whole group, and a sub-presentation for the subgroup generated by the gate set named in the Object column of the Circuits block. L_n is the group of orthogonal matrices $N/\sqrt{2}^j$, where N is an integer matrix and $j \geq 0$. Mod counts Agda modules. The Agda times are typechecking times from scratch with Agda 2.8, the times its profiler reports for the row's modules; the Lean times are elapsed folder-build times. Both were measured on the same machine, under the conditions of Section 3.3. Both columns exclude the shared library and other developments on which a row depends. The Result column describes the Agda statements; Section 3.3 explains the semantic differences. *Includes two unused legacy helpers that failed; the main theorem chain passed.

numbers, no such automatic check exists, which limits the equality checking of QCert to discrete gate sets, as we discuss in Section 7.

In the Agda development, most theorems in Table 1 are proved from scratch in QCert against a matrix semantics, with three exceptions. The two rows of [10, 11] build on the Agda proofs that accompany those papers. These proofs relate the circuit relations to two-level relations by the Reidemeister–Schreier method [42]. We connect them to our formalizations of the two-level matrix presentations in [9, 31], the first two rows of the table, to obtain sub-presentations of

$U_4\left(\mathbb{Z}\left[1/\sqrt{2}, i\right]\right)$ and $U_8\left(\mathbb{Z}\left[1/2, i\right]\right)$. The qudit Clifford theorem of Bian, Li et al. [8] is the only one whose semantics is not a matrix group. Its group presentation lifts a presentation of the symplectic group $\mathrm{Sp}(2n, \mathbb{Z}_p)$ with Pauli corrections and global phases, and its semantics is built in the same way (Section 4).

6 Corrections to the Source Papers

The formalizations in Table 1 identify two kinds of issues in the source papers. Most are typos, and some of them appear in statements that the proofs rely on. These are easy to fix. Two others are harder to repair. One is a gap in a proof, and the other is a relation missing from a theorem. We describe both below.

The gap is in the proof of Theorem 4.14 of Hadamard-Π [26], which states that the relations in its Figure 6, over the one- and two-level matrices $(-1)_{[a]}$, $\mathrm{X}_{[a,b]}$, and $\mathrm{H}_{[a,b]}$, present the group $O_n\left(\mathbb{Z}\left[1/\sqrt{2}\right]\right)$. One case of the proof relies on Lemmas A.16 and A.17, which have counterexamples in dimension 6, and the case analysis built on them misses a matrix in dimension 9. The authors have confirmed this gap (personal communication). Their main result is however still correct: Clément's unpublished notes [19] present the same group with different relations, and his case analysis covers the missing case, using which the formalization of Theorem 4.14 is completed. We also prove that the two sets of relations are equivalent in every dimension and formalize Clément's proof [4].

The relation is missing from Theorem 5.2 of [3]. It states that the relations in its Figures 4 and 7, over the Toffoli and $\mathrm{K} = \mathrm{H} \otimes \mathrm{H}$ gates, present the group $O_8\left(\mathbb{Z}\left[1/2\right]\right)$. This is the first of two steps towards presenting the scaled dyadic orthogonal group L_8 by three-qubit Toffoli–Hadamard circuits. The relation r_{K} in its Section 5.5, which splits K on two qubits into two copies controlled by the third qubit, is in neither figure. Nor can it be derived from them. Hence, Theorem 5.2 fails as stated, and we have informed the authors. Our formalization [5] shows that r_{K} is independent by giving an interpretation of circuits that every other relation preserves but r_{K} does not, and proves the presentations of $O_8\left(\mathbb{Z}\left[1/2\right]\right)$ and L_8 with r_{K} restored.

7 Discussion and Future Work

We formalize the results of sixteen papers in QCert, using consistent circuit syntax, a unified definition of group presentations, and shared libraries for rings and their matrix groups. Since all formalizations use these definitions, presentations of the same group from different papers can be compared and proved equivalent. The same libraries can be reused for new algorithms, and the induction that turns an exact synthesis algorithm into a completeness proof carries over with little change. The main part specific to each algorithm is the case analysis over matrices and generators. As the most error-prone part of the proof, it is where machine checking in QCert matters most.

At the moment, not every completeness result fits into QCert, as its scope is limited in two ways. First, it supports only discrete gate sets, so it does not cover completeness results for circuits with continuous parameters, such as rotations by arbitrary angles [21–23]. Second, control is not a circuit constructor in QCert, so it does not cover results that treat control as a primitive operation on circuits [12, 25, 51].

We plan to extend the infrastructure of QCert to more families of quantum circuits, including the two above. Completeness is still open for many discrete gate sets in higher-dimensional quantum systems, and their case analyses are often too large to check by hand. In particular, we plan to study the completeness of Clifford circuits in all dimensions with the help of QCert. Beyond completeness,

the exact synthesis libraries can be reused for T-count optimization of multi-qubit Clifford+T circuits.

References

[1] Matthew Amy, Jianxin Chen, and Neil J. Ross. 2018. A finite presentation of CNOT-dihedral operators. In *Proceedings of the 14th International Conference on Quantum Physics and Logic (QPL 2017) (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*. 84–97. doi:10.4204/EPTCS.266.5

[2] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. 2020. Number-theoretic characterizations of some restricted Clifford+T circuits. *Quantum* 4 (2020), 252.

[3] Matthew Amy, Neil J. Ross, and Scott Wesley. 2024. A sound and complete equational theory for 3-qubit Toffoli-Hadamard circuits. In *Proceedings of the 21st International Conference on Quantum Physics and Logic (QPL 2024) (Electronic Proceedings in Theoretical Computer Science, Vol. 406)*. 1–43. arXiv:2407.11152 doi:10.4204/EPTCS.406.1

[4] Anonymous. 2026. QCert: The presentations of $O_n(\mathbb{Z}[1/\sqrt{2}])$ by Hadamard-Π and by Clément, formalized in Agda. Supplementary material, modules `Real-Clifford+CH-TwoLevel/Equivalence` (the equivalence of the two sets of relations) and `Real-Clifford+CH-TwoLevel-Clement` (Clément's proof and its decision trees).

[5] Anonymous. 2026. QCert: Three-qubit Toffoli–Hadamard circuits with the relation r_{K}, formalized in Agda. Supplementary material, modules `CCX+HH-3qubit-opus/Theorems` (the presentations of $O_8(\mathbb{Z}[1/2])$ and L_8) and `CCX+HH-3qubit-opus/Independence` (the interpretation showing that r_{K} is not derivable).

[6] Alexander Bastounis, Fabian Circelli, and Anders C. Hansen. 2026. Navier-Stokes lost in translation: Why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. *arXiv preprint arXiv:2610.08144* (2026).

[7] Xiaoning Bian and Yuan Feng. 2027. Normal forms and complete relations for circuits in Agda. *Proceedings of the ACM on Programming Languages* POPL (2027). To appear.

[8] Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026. A complete and natural rule set for multi-qudit Clifford circuits in all odd prime dimensions. arXiv:2609.40106

[9] Xiaoning Bian and Peter Selinger. 2021. Generators and relations for $U_n(\mathbb{Z}[1/2, i])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 145–164. arXiv:2105.14047 doi:10.4204/EPTCS.343.8

[10] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 2-qubit Clifford+T operators. In *Proceedings of the 19th International Conference on Quantum Physics and Logic (QPL 2022) (Electronic Proceedings in Theoretical Computer Science, Vol. 394)*. 13–28. arXiv:2204.02217 doi:10.4204/EPTCS.394.2

[11] Xiaoning Bian and Peter Selinger. 2023. Generators and relations for 3-qubit Clifford+CS operators. In *Proceedings of the 20th International Conference on Quantum Physics and Logic (QPL 2023) (Electronic Proceedings in Theoretical Computer Science, Vol. 384)*. 114–126. arXiv:2306.08530 doi:10.4204/EPTCS.384.7

[12] Colin Blake. 2026. A complete equational presentation of qudit circuits via polycontrolled PROPs. In *Proceedings of the 51st International Symposium on Mathematical Foundations of Computer Science (MFCS 2026) (Leibniz International Proceedings in Informatics, Vol. 386)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 6:1–6:18. arXiv:2602.09873 doi:10.4230/LIPIcs.MFCS.2026.6

[13] Colin Blake. 2026. Completeness for prime-dimensional phase-affine circuits. arXiv:2603.06466

[14] Colin Blake. 2026. Simpler presentations for many fragments of quantum circuits. In *Proceedings of the 11th International Conference on Formal Structures for Computation and Deduction (FSCD 2026) (Leibniz International Proceedings in Informatics, Vol. 378)*. 6:1–6:20. arXiv:2602.09874 doi:10.4230/LIPIcs.FSCD.2026.6

[15] Ana Bove, Peter Dybjer, and Ulf Norell. 2009. A brief overview of Agda – a functional language with dependent types. In *Theorem Proving in Higher Order Logics (TPHOLs 2009) (Lecture Notes in Computer Science, Vol. 5674)*. Springer, Berlin, Heidelberg, 73–78.

[16] Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. With a few square roots, quantum computing is as easy as Π. *Proceedings of the ACM on Programming Languages* 8, POPL (2024). arXiv:2310.14056

[17] Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An automated deductive verification framework for circuit-building quantum programs. In *Programming Languages and Systems: 30th European Symposium on Programming (ESOP 2021) (Lecture Notes in Computer Science, Vol. 12648)*. 148–177.

[18] Vikraman Choudhury, Jacek Karwowski, and Amr Sabry. 2022. Symmetries in reversible programming: From symmetric rig groupoids to reversible programming languages. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 6 (2022), 32 pages.

[19] Alexandre Clément. 2019. A complete set of relations for the real Clifford+T group. Unfinished manuscript. https://www.mathstat.dal.ca/~aclement/Case%20distinction.pdf

[20] Alexandre Clément. 2026. A complete equational theory for real-Clifford+CH quantum circuits. arXiv:2602.06644

[21] Alexandre Clément, Noé Delorme, and Simon Perdrix. 2024. Minimal equational theories for quantum circuits. In *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2024)*. 27:1–27:14. arXiv:2311.07476 doi:10.1145/3661814.3662088

[22] Alexandre Clément, Noé Delorme, Simon Perdrix, and Renaud Vilmart. 2024. Quantum circuit completeness: Extensions and simplifications. In *Proceedings of the 32nd EACSL Annual Conference on Computer Science Logic (CSL 2024) (Leibniz International Proceedings in Informatics, Vol. 288)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 20:1–20:23. arXiv:2303.03117 doi:10.4230/LIPIcs.CSL.2024.20

[23] Alexandre Clément, Nicolas Heurtel, Shane Mansfield, Simon Perdrix, and Benoît Valiron. 2023. A complete equational theory for quantum circuits. In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2023)*. 1–13. arXiv:2206.10577 doi:10.1109/LICS56636.2023.10175801

[24] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*. Springer, 625–635. doi:10.1007/978-3-030-79876-5_37

[25] Noé Delorme and Simon Perdrix. 2026. Diagrammatic reasoning with control as a constructor, applications to quantum circuits. In *Foundations of Software Science and Computation Structures (FoSSaCS 2026) (Lecture Notes in Computer Science, Vol. 16503)*. Springer Nature Switzerland, Cham, Switzerland, 240–261. arXiv:2508.21756 doi:10.1007/978-3-032-22730-0_12

[26] Wang Fang, Chris Heunen, and Robin Kaarsgaard. 2026. Hadamard-Π: Equational quantum programming. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 5 (2026), 27 pages. arXiv:2506.06835 doi:10.1145/3776647

[27] Shiguang Feng and Lvzhou Li. 2026. A complete set of transformation rules for reversible circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 8 (2026), 3711–3724. arXiv:2508.17273 doi:10.1109/TCAD.2025.3641533

[28] Shiguang Feng, Zongxing Xiong, and Lvzhou Li. 2026. On the completeness of transformation rules in reversible logic synthesis. arXiv:2607.19059

[29] Brett Giles and Peter Selinger. 2013. Exact synthesis of multiqubit Clifford+T circuits. *Physical Review A* 87 (2013), 032332. arXiv:1212.0506

[30] Andrew N. Glaudell, Neil J. Ross, and Jacob M. Taylor. 2021. Optimal two-qubit circuits for universal fault-tolerant quantum computation. *npj Quantum Information* 7 (2021), 103. arXiv:2001.05997 doi:10.1038/s41534-021-00424-z

[31] Seth Eveson Murray Greylyn. 2014. *Generators and relations for the group* $U_4(\mathbb{Z}[1/\sqrt{2}, i])$. MSc thesis. Dalhousie University. arXiv:1408.6204

[32] David Gross. 2006. Hudson's theorem for finite-dimensional quantum systems. *J. Math. Phys.* 47, 12 (2006), 122107. doi:10.1063/1.2393152

[33] Shamgar Gurevich and Ronny Hadani. 2007. The geometric Weil representation. *Selecta Mathematica* 13 (2007), 465–481. doi:10.1007/s00029-007-0047-3

[34] Shamgar Gurevich and Ronny Hadani. 2010. Bounds on certain higher-dimensional exponential sums via the self-reducibility of the Weil representation. arXiv:math/0612765 [math.RT] https://arxiv.org/abs/math/0612765

[35] Thomas Hales. 2026. What mathematicians should know about the Lean theorem prover: Questions of reliability and AI. Guest post on Terence Tao's blog *What's new*. https://terrytao.wordpress.com/2026/10/09/what-mathematicians-should-know-about-the-lean-theorem-proverquestions-of-reliability-and-ai/ 9 October 2026.

[36] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 37 (2021), 29 pages. doi:10.1145/3434318

[37] Yves Lafont. 2003. Towards an algebraic theory of Boolean circuits. *Journal of Pure and Applied Algebra* 184, 2–3 (2003), 257–310. doi:10.1016/S0022-4049(03)00069-0

[38] Sarah Meng Li, Michele Mosca, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2025. A complete and natural rule set for multi-qutrit Clifford circuits. In *Proceedings of the 22nd International Conference on Quantum Physics and Logic (QPL 2025) (Electronic Proceedings in Theoretical Computer Science, Vol. 426)*. 23–78. arXiv:2508.14670 doi:10.4204/EPTCS.426.2

[39] Sarah Meng Li, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for the group $O_n(\mathbb{Z}[1/2])$. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 210–264. arXiv:2106.01175 doi:10.4204/EPTCS.343.11

[40] Sirui Lu, Ruixuan Deng, Yanqiao Zhu, and Zhengfeng Ji. 2026. Long-horizon autoformalization of a core theorem underlying MIP* = RE. arXiv:2609.19814 [quant-ph] doi:10.48550/arXiv.2609.19814

[41] Saunders Mac Lane. 1965. Categorical algebra. *Bull. Amer. Math. Soc.* 71, 1 (1965), 40–106. doi:10.1090/S0002-9904-1965-11234-4

[42] Wilhelm Magnus, Abraham Karrass, and Donald Solitar. 2004. *Combinatorial group theory: Presentations of groups in terms of generators and relations* (2nd revised ed.). Dover Publications.

[43] Justin Makary, Neil J. Ross, and Peter Selinger. 2021. Generators and relations for real stabilizer operators. In *Proceedings of the 18th International Conference on Quantum Physics and Logic (QPL 2021) (Electronic Proceedings in Theoretical Computer Science, Vol. 343)*. 14–36. arXiv:2109.05655 doi:10.4204/EPTCS.343.2

[44] Dmitri Maslov, Gerhard W. Dueck, D. Michael Miller, and Camille Negrevergne. 2008. Quantum circuit simplification and level compaction. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 27, 3 (2008), 436–444. doi:10.1109/TCAD.2007.911334

[45] Yunseong Nam, Neil J. Ross, Yuan Su, Andrew M. Childs, and Dmitri Maslov. 2018. Automated optimization of large quantum circuits with continuous parameters. *npj Quantum Information* 4 (2018), 23. doi:10.1038/s41534-018-0072-4

[46] Markus Neuhauser. 2002. An Explicit Construction of the Metaplectic Representation over a Finite Field. *Journal of Lie Theory* 12, 1 (2002), 15–30. doi:10.5802/jolt.250

[47] Ulf Norell. 2007. *Towards a practical programming language based on dependent type theory*. Ph. D. Dissertation. Chalmers University of Technology.

[48] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A core language for quantum circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*. 846–858.

[49] Yuanjie (Collin) Ren, Zhicheng Zhang, Junyi Liu, Jinzheng Li, Yidi Qi, and Mingsheng Ying. 2026. QEDIA: Quantum program verification in Lean. arXiv:2610.17708 [quant-ph] doi:10.48550/arXiv.2610.17708

[50] Issei Sakashita. 2013. A complete transformation rule set and a minimal equation set for CNOT-based 3-qubit quantum circuits (Draft). arXiv:1304.5916

[51] William Schober and Scott Wesley. 2026. A complete equational theory for quantum circuits with generalized control. https://cqi.inf.usi.ch/publications/Hierarchical_QPL2026.pdf Accepted to the proceedings track of the 23rd International Conference on Quantum Physics and Logic (QPL 2026).

[52] Peter Selinger. 2015. Generators and relations for n-qubit Clifford operators. *Logical Methods in Computer Science* 11, 2:10 (2015). arXiv:1310.6813 doi:10.2168/LMCS-11(2:10)2015

[53] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. t|ket⟩: A retargetable compiler for NISQ devices. *Quantum Science and Technology* 6, 1 (2020), 014003. doi:10.1088/2058-9565/ab8e92

[54] The mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. 367–381. doi:10.1145/3372885.3373824

[55] Mingkuan Xu, Zikun Li, Oded Padon, Sina Lin, Jessica Pointing, Auguste Hirth, Henry Ma, Jens Palsberg, Alex Aiken, Umut A. Acar, and Zhihao Jia. 2022. Quartz: Superoptimization of quantum circuits. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, 625–640. doi:10.1145/3519939.3523433

[56] Shigeru Yamashita and Igor L. Markov. 2010. Fast equivalence-checking for quantum circuits. *Quantum Information and Computation* 10, 9–10 (2010), 721–734. doi:10.26421/QIC10.9-10-1

[57] Lei Zhang, Mingrui Jing, Yusheng Zhao, Jiahong Liu, and Xin Wang. 2026. Lean-QuantumAlg: A framework for AI agents to produce quantum algorithms that humans can trust. arXiv:2610.30216 [quant-ph] doi:10.48550/arXiv.2610.30216

[58] Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. 2023. CoqQ: Foundational verification of quantum programs. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 29 (2023), 33 pages.

A Agda and Lean Comparison

	Agda	Lean
Proof development		
Logic	constructive: every module `--safe`, no postulates	classical: every main theorem uses `Classical.choice`
Library	standard library (105 modules imported), EucDomain	Mathlib (56 modules imported directly)
Proofs	terms, by pattern matching and equational reasoning; reflection-based solvers	tactics: about 75,000 tactic blocks, 69,000 automation calls
Reused from the sources	the authors' Agda proofs for Clifford+T and Clifford+CS	no proofs; the authors' coset-action data for Clifford+T
Modules	1,343	795
Code lines	355,086	338,866
Verification time		
Decided equalities	a Boolean reduced to `true`	`decide +kernel`
Check time, sixteen rows	2,399 s	9,305 s
Faster for	thirteen rows	three rows
Execution time		
Normal forms	executable functions	existence theorems or `noncomputable` definitions (460 declared)
Benchmarked procedure	matrix of each circuit, by products of stored matrices	simulation of each circuit on every basis vector
Compiled through	GHC (Haskell)	C
Per decision, Clifford, 100 gates	5.0 ms on 2 qubits, 24 s on 6	5.8 ms on 2 qubits, 0.61 s on 6

Table 2. The Agda and Lean formalizations compared. Counts are over the files under version control in the sixteen development folders of Table 1, without the shared libraries, and code lines exclude blank lines and comments. Check times are those of Table 1, and execution times those of Figure 6(c).